École Supérieure de Technologie de Guelmim

Software Defined Networking Architecture

Study, Design, and Implementation

Réalisé par :

Abderrahmane Aroussi

Amin Mriroud

Encadrant : Pr. Tarek AIT BAHA

Program: Computer Networks & Security (RIS)

Jury : Pr. Amine Bouaouda • Pr. Tarek AIT BAHA

Academic Year 2025–2026

Why SDN? — Our Project Story

✕ Initial Project

Real-Time Detection & Analysis Platform for Local Network Security

- Topic already covered by multiple groups in the same RIS program
- Similar approaches — risk of repetition & lack of added value
- Missed opportunity to stand out technically

✓ Chosen Project

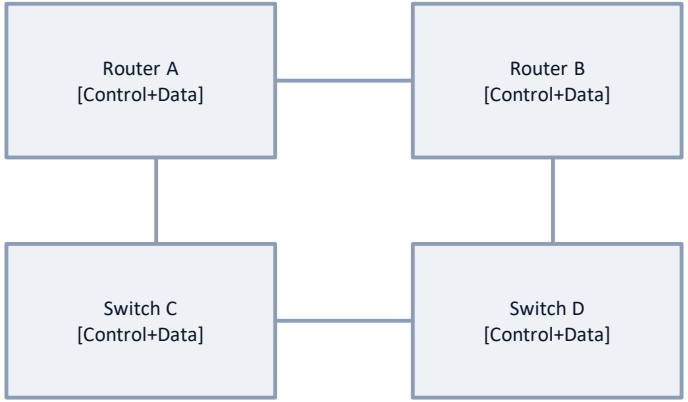
SDN Architecture (OpenDaylight + Mininet)

- Suggested by our supervisor Pr. Tarek AIT BAHA
- Emerging technology — SDN not taught in our RIS curriculum
- Strong technical challenge & unique topic = real added value

We chose to go beyond our curriculum — and implement what we had never been taught.

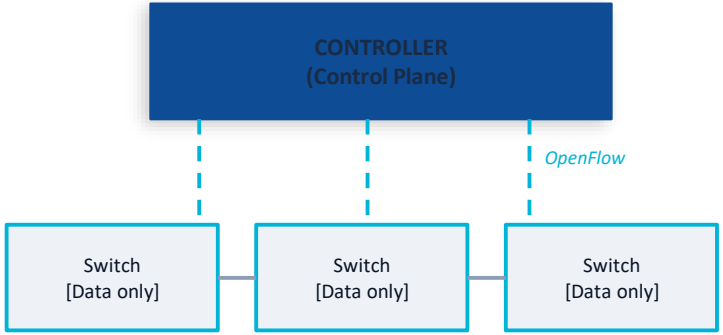
Traditional Networks vs SDN

TRADITIONAL NETWORK



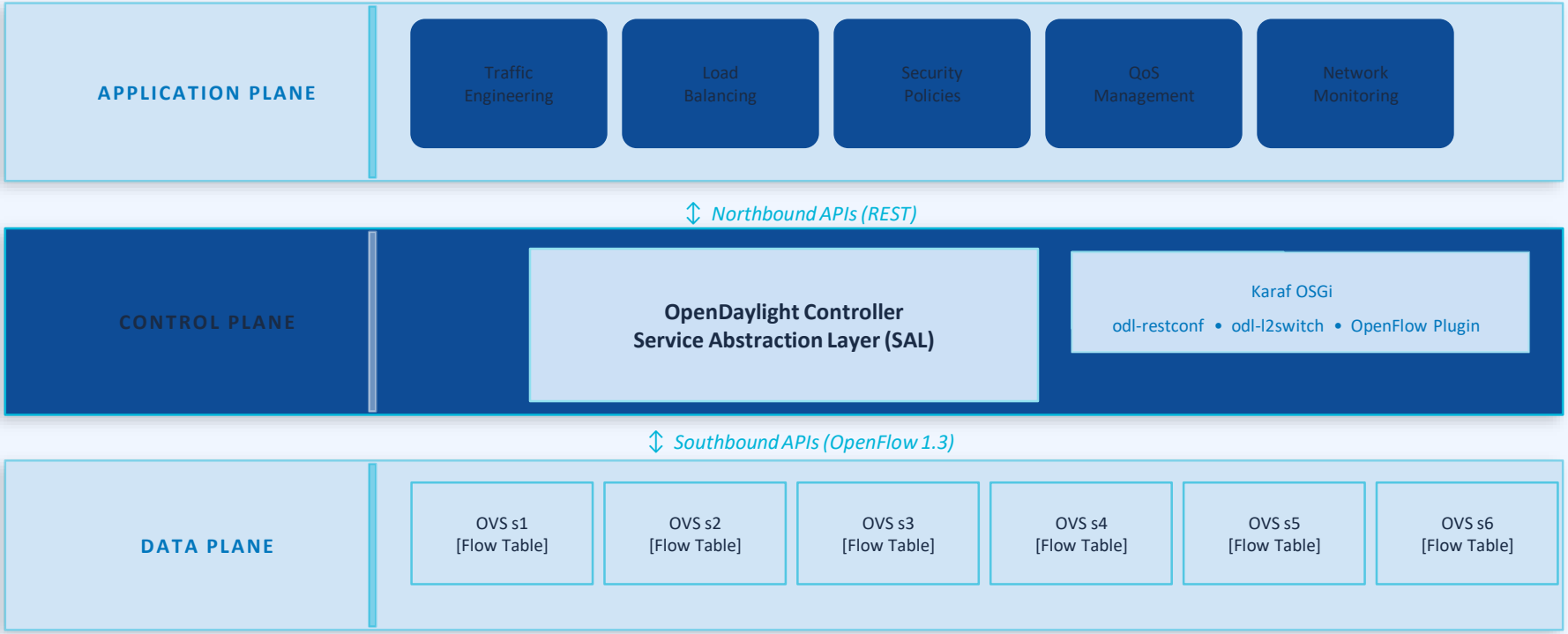
- ✗ Each device makes its own decisions
- ✗ Hard to manage at scale
- ✗ No global view of the network
- ✗ Slow to adapt to changes

SDN — SOFTWARE DEFINED



- ✓ Centralized intelligence & global view
- ✓ Programmable routing rules (OpenFlow)
- ✓ Dynamic adaptation to traffic changes
- ✓ Scalable & easier to manage

SDN Architecture — Three Layers



OpenFlow Protocol — How It Works

Flow Entry Structure

Match Fields
MAC src/dst, IP, ports, protocol

Priority
Rule priority (0 = lowest)

Counters
n_packets, n_bytes matched

Instructions
Forward / Drop / Send to Controller

Timeouts
idle_timeout / hard_timeout (seconds)

From our experiment (switch s1):

```
dl_src=00:00:00:01 → output:s1-eth2  |  priority=10  |
n_packets=41
```

Reactive Flow: Learning Phase

1. Host A sends packet
   → Switch has NO rule
2. PACKET_IN sent to controller
   (unknown destination)
3. Controller analyzes & installs
   FLOW_MOD in switch table
4. Switch forwards packet
   All future packets → forwarded directly

Result: 50% loss (1st pingall) → 0% loss (2nd pingall)

Experimental Environment & Tools

OpenDaylight

SDN Controller

Java/OSGi • REST APIs
odl-restconf + odl-l2switch

Docker

Containerization

ODL runs in isolated container
Reproducible environment

Mininet 2.3.0

Network Emulator

Virtual switches & hosts
Linux network namespaces

OpenFlow 1.3

Southbound Protocol

Controller ↔ Switch comms
Flow table management

iPerf3

Performance Testing

TCP throughput measurement
between emulated hosts

Wireshark

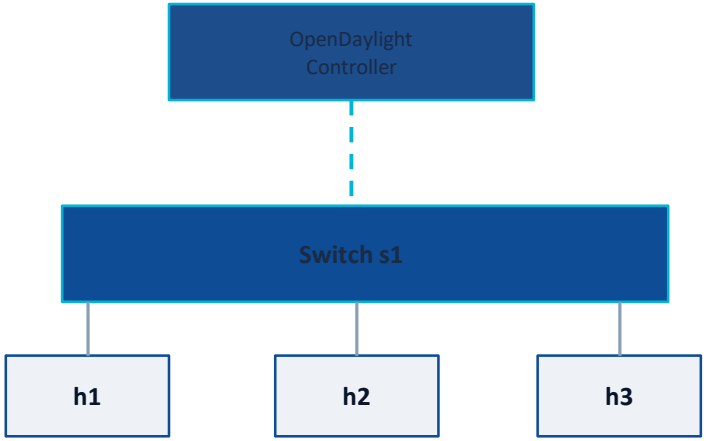
Protocol Analysis

OpenFlow traffic capture
PACKET_IN / FLOW_MOD analysis

Host Machine: Ubuntu 24.04 VM • 4 CPU cores • 10 GB RAM

Network Topologies Tested

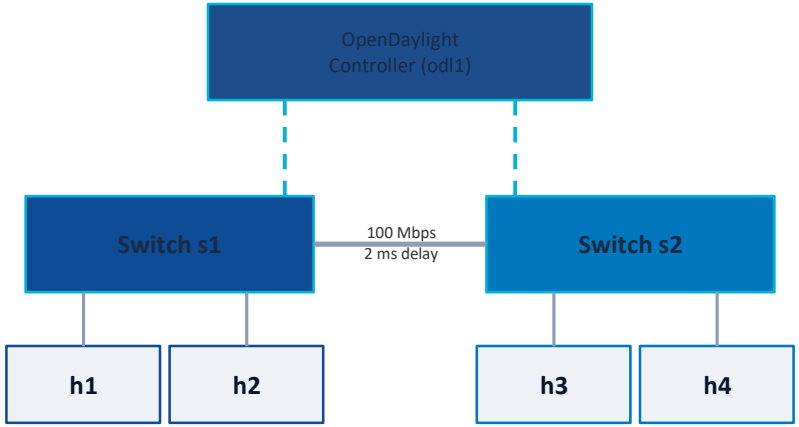
Topology 1 — Single Switch



Mininet command:

```
sudo mn --controller=remote,
  ip=127.0.0.1,port=6633 --topo=single,3
```

Topology 2 — Dual Switch



Custom Python topology script:

```
sudo mn --custom projet_sdn.py
  --topo projet_sdn --controller=remote
```

Results — Connectivity Tests

1st pingall — Learning Phase

```
mininet> pingall
h1 -> X X
h2 -> X h3
h3 -> h1 h2
*** Results: 50% dropped (3/6 received)
```

Switches query controller for unknown flows → PACKET_IN

2nd pingall — Flows Installed ✓

```
mininet> pingall
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
```

Controller installed FLOW_MOD → all subsequent packets forwarded directly

Wireshark Analysis — OpenFlow Messages Captured

OFPT_HELLO

Controller ↔ switch handshake

OFPT_FEATURES_REQUEST

Controller queries switch capabilities

OFPT_PACKET_IN

Switch sends unknown packet to controller

OFPT_FLOW_MOD

Controller installs flow rules in switch

OFPT_MULTIPART_REQUEST

Controller requests statistics

Results — Throughput Performance (iPerf)

640
Mbit/s
h1 → h2 (same switch)
Intra-switch — no inter-link overhead

585
Mbit/s
h1 → h4 (diff. switches)
Inter-switch — traverses 2 hops

9.81
Gbits/s
Multi-controller topology
6 switches / no BW limit imposed

iPerf Test Commands

```
mininet> h4 iperf -s &
mininet> h1 iperf -c 10.0.0.4
```

Result interpretation

The 640 → 585 Mbit/s difference confirms that the additional hop (inter-switch link) introduces processing overhead — expected SDN behavior.

Flow Table Inspection (dpctl dump-flows)

```
Match: dl_src=00:00:00:00:00:01, dl_dst=00:00:00:00:00:02  |  Action: output:s1-eth2  |  Priority: 10 | n_packets: 41
Match: dl_src=00:00:00:00:00:02, dl_dst=00:00:00:00:00:01  |  Action: output:s1-eth1  |  Priority: 10 | n_packets: 2
Match: in_port=s1-eth1 (flooding)  |  Action: output:s1-eth2, CONTROLLER  |  Priority: 2
Match: * (catch-all)  |  Action: drop  |  Priority: 0
```

Multi-Controller Experiment — Scalability

6 Switches × 3 Controllers × 12 Hosts

```
*** Results: 0% dropped
(132/132 received)

RTT h1→h2:  0.075 / 0.151 ms
RTT h1→h6:  0.067 / 0.276 ms
RTT h1→h12: 0.080 / 0.185 ms
iPerf h1→h6: 9.81 Gbits/sec
```

Key Findings & Significance

Full connectivity

0% packet loss across all 132 host pairs — 3 controllers working in harmony

High throughput

9.81 Gbits/sec — control plane overhead negligible once flows established

Low latency

RTT < 0.3 ms across the network — even for h1→h12 (6 hops)

Scalability proven

From 2 switches to 6 — architecture scaled without stability issues

Context vs literature

Khattak et al. (2014) reported 55 req/s — our modern ODL shows major improvement

SDN in the Real World — Applications

Production

Google B4

Wide-Area Networks

SDN used for traffic engineering across Google's global backbone — 2× bandwidth utilization improvement

Research

SDN-VANET

Vehicular Networks

Centralized controller manages RSUs & vehicle comms — dynamic traffic management, low-latency safety alerts

Research

OpenSAN

Satellite Networks

SDN control plane over satellite ground stations — virtualization, mobility & resource management for vast infra

SDN-Based UAV Networks

Research

A centralized controller maintains logical control links to each UAV (drone). Enables dynamic path planning, coordinated flight & adaptive communication.
Our multi-controller experiment with 6 switches is a direct stepping stone toward this architecture.

Our prototype → foundational model for all three: centralized controller + OpenFlow + scalable multi-switch topology

Challenges Encountered & Solutions

Container Issue

⚠ ODL Container Startup Error

shell.init.script: String index out of range: 0 — Karaf failed to start

→

✓ Recreated container in interactive mode (-itd)
Manually installed features via Karaf CLI

OS Compatibility

⚠ MiniEdit Compatibility Error

Python 3 import failure — StrictVersion not found (distutils removed in Ubuntu 24.04)

→

✓ Switched to custom Python topology scripts
All experiments completed via CLI — no GUI needed

SDN Behavior

⚠ Initial 50% Packet Loss

PACKET_IN flood during learning phase — all unknown flows sent to controller

→

✓ Expected SDN behavior — understood & documented
Confirmed 0% loss after flow rules installed

Advanced Setup

⚠ Multi-Controller Coordination

Three independent ODL instances on Docker bridge — port conflicts, switch assignment

→

✓ Mapped to distinct ports (6633/6634/6635)
Custom Python script assigned each switch to a controller

Conclusion & Future Perspectives

What We Achieved

- Deployed OpenDaylight inside Docker — reproducible environment
- Emulated single/dual/multi-switch topologies with Mininet
- Validated 0% packet loss after OpenFlow learning phase
- Measured 640 Mbit/s (intra) / 585 Mbit/s (inter-switch) throughput
- Extended to 3 controllers × 6 switches × 12 hosts — 9.81 Gbits/s
- Analyzed OpenFlow traffic with Wireshark (PACKET_IN, FLOW_MOD)

Future Work

- Cbench benchmarking for quantitative controller evaluation
- SDN-based security: DDoS mitigation, distributed firewall
- East-west protocols for multi-controller state synchronization
- QoS measurements: latency, jitter, loss under diverse loads
- Kubernetes integration — SDN network policies for microservices

"We chose to learn what was not taught — and built a working SDN prototype from scratch."
— Abderrahmane Aroussi & Amin Mriroud, 2026